IBM Consulting Advantage

Developer API Overview

LLM chat, retrieval (RAG) and MCP tool endpoints

Prepared by Dusan Salovic · 23 October 2026 · Source: ICA Developer API Swagger UI (OpenAPI 3.0)

1. At a glance

The IBM Consulting Advantage (ICA) Developer API is a REST API for using ICA's language models, assistants, agents and digital workers from your own code. The chat and model-listing endpoints follow the OpenAI API format, so existing OpenAI clients and SDKs can be pointed at it with little change.

Item	Value
Base URL	`https://api.servicesessentials.ibm.com/v1`
Specification	OpenAPI 3.0 (openapi.yaml, served next to the Swagger UI)
Authentication	Bearer token: developer API key in the Authorization header
Compatibility	OpenAI chat.completion and models.list shapes; MCP streamable-HTTP for tools
Resource groups	7: Agents, Assistants, Chat Models, Digital Workforce, Document Collections, Files, Tools
Operations	29 in total (12 chat/listing, 10 file and collection, 7 MCP tool server)
Data models	23 schemas
Built on	Per the Tools descriptions: IBM ContextForge as the MCP gateway, with tool servers also connected to OpenWebUI

The API covers three capabilities:

- **Chat.** OpenAI-style chat completions against four kinds of targets (raw models, assistants, agents, digital workers), all with the same request and response shape.
- **Retrieval (RAG).** Upload files, group them into document collections, and reference either one in a chat request. Responses then include the source chunks used.
- **Tools (MCP).** Register, search and connect to Model Context Protocol tool servers through a managed gateway, using any standard MCP client.

2. Authentication and errors

Every request must carry your developer API key as an OpenAI-style bearer token:

```
Authorization: Bearer <ICA_DEVELOPER_API_KEY>
```

Keys are issued in the ICA UI under **Settings → API Keys → ICA APIs**. Entitlements are checked per resource, so a valid key can still get a 403 for a model, assistant or agent it is not entitled to.

All errors return a JSON body with a single detail field:

```
{ "detail": "string" }
```

Status	Meaning
400	Validation error (malformed request body or parameters).
401	Missing, unknown or revoked API key.
403	Key is valid but not entitled to this resource, or (for tools) the server is not owned by your team.
404	Resource not found.
502	POST /tools only: the upstream MCP server or ContextForge failed to provision the gateway.

3. Chat endpoints

Chat-style endpoints are exposed under four namespaces. They share the same request and response shape and differ only in what the model ID points to.

Namespace	What you are talking to
`/chat-models`	Raw foundation models (LLMs).
`/assistants`	Assistants configured in ICA.
`/agents`	Agents available in ICA.
`/digital-workforce`	Digital workers from the ICA catalogue.

Each namespace offers the same three operations:

Operation	Purpose
`GET /{namespace}`	List available items. Friendly alias with the same response as /models.
`GET /{namespace}/models`	OpenAI-compatible models.list. Use the returned id as the model in a chat request.
`POST /{namespace}/chat/completions`	Run a chat completion.

3.1 Endpoint list

Method	Path	Purpose
GET	`/chat-models`	List chat models
GET	`/chat-models/models`	List chat models (OpenAI-compatible)
POST	`/chat-models/chat/completions`	Chat model completion
GET	`/assistants`	List assistants
GET	`/assistants/models`	List assistants (OpenAI-compatible)
POST	`/assistants/chat/completions`	Assistant chat completion
GET	`/agents`	List agents
GET	`/agents/models`	List agents (OpenAI-compatible)
POST	`/agents/chat/completions`	Agent chat completion
GET	`/digital-workforce`	List digital workers
GET	`/digital-workforce/models`	List digital workers (OpenAI-compatible)

Method	Path	Purpose
POST	/digital-workforce/chat/completions	Digital worker chat completion

> **Practical tip:** each namespace exposes /models and /chat/completions, so each one can act as the base URL of a standard OpenAI SDK client (for example base_url = …/v1/chat-models). Moving from a raw model to an assistant or agent then only means changing the base URL and model ID.

3.2 Request: ChatCompletionRequest

Field	Required	Notes
model	**Yes**	ID from GET /{namespace}/models, e.g. "agent-finance-helper".
messages	**Yes**	Array of ChatMessage (see below).
stream	No	Stream the response. Examples use false.
temperature	No	Sampling temperature.
top_p	No	Nucleus sampling.
max_tokens	No	Maximum tokens to generate.
files	No	Array of FileRef to ground the answer in files and/or collections. Both types can be mixed.

ChatMessage and FileRef:

Field	Required	Notes
ChatMessage.role	**Yes**	Message role (system, user, assistant).
ChatMessage.content	**Yes**	Message text.
FileRef.type	**Yes**	"file" or "collection".
FileRef.id	**Yes**	File ID or collection ID.
FileRef.name	No	Display name.
FileRef.context	No	Additional context for the reference.

> **Note:** the request schema has no tools, tool_choice or response_format fields. OpenAI-style function calling and structured output are therefore not exposed through chat completions. Tool use happens either inside ICA agents and assistants, or through the MCP endpoints in section 5.

3.3 Response: ChatCompletionResponse

Field	Required	Notes
id	No	Completion ID, e.g. "chatcmpl-abc123".
object	No	"chat.completion".
created	No	Unix timestamp.
model	No	Model that served the request.
choices[]	No	ChatCompletionChoice: index, finish_reason (e.g. "stop"), message (ChatMessage).
usage	No	prompt_tokens, completion_tokens, total_tokens.

Field	Required	Notes
sources[]	No	ICA extension. Retrieved chunks used to ground the answer; only present when files or collections were attached. Each Source has source.name, document[] (text chunks) and metadata[].

3.4 Example

```
POST https://api.servicesessentials.ibm.com/v1/agents/chat/completions
Authorization: Bearer <ICA_DEVELOPER_API_KEY>
Content-Type: application/json

{
  "model": "agent-finance-helper",
  "messages": [ { "role": "user", "content": "Summarise our Q1 spend." } ],
  "stream": false,
  "files": [ { "type": "collection", "id": "<collection-id>" } ]
}
```

Response:

```
{
  "id": "chatcmpl-abc123",
  "object": "chat.completion",
  "created": 1715000000,
  "model": "agent-finance-helper",
  "choices": [ { "index": 0, "finish_reason": "stop",
                 "message": { "role": "assistant", "content": "…" } } ],
  "usage": { "prompt_tokens": 0, "completion_tokens": 0, "total_tokens": 0 },
  "sources": [ { "source": { "name": "q1-report.pdf" },
                 "document": [ "…chunk text…" ], "metadata": [ { } ] } ]
}
```

3.5 Model listing: ModelsResponse

All listing endpoints return the OpenAI models.list shape. Each Model contains:

Field	Required	Notes
id	No	Value to pass as model in chat requests.
name	No	Display name.
object	No	"model".
created	No	Unix timestamp.
owned_by	No	"ica".
info.meta	No	description and tags[]. Useful for building a model or agent picker.

4. Files and document collections (RAG)

Retrieval-augmented generation is built in. Documents are uploaded as files and then used in one of two ways:

Option	Reference in chat	Use when
Single file	`{ "type": "file", "id": "<file-id>" }`	You want to ask about one or a few specific documents.
Collection	`{ "type": "collection", "id": "<collection-id>" }`	You want a reusable knowledge base searched across many files.

4.1 Typical flow

1. Upload the document with `POST /files` (multipart/form-data, field `file`). The file record is returned immediately with `data.status = "pending"`.
2. Poll `GET /files/{file_id}` until `data.status == "completed"`. Text extraction runs asynchronously and must finish before the file can be used.
3. Either reference the file directly in chat, or create a collection (`POST /document-collections`) and attach the file (`POST /document-collections/{id}/file/add`).
4. Pass the file or collection in the `files` array of a chat completion. The answer comes back with `sources`.

4.2 Files

Method	Path	Purpose
`POST`	`/files`	Upload a file (multipart field "file"). Returns the FileObject.
`GET`	`/files`	List your files (array of FileObject).
`GET`	`/files/{file_id}`	Get file metadata, including extraction status.
`DELETE`	`/files/{file_id}`	Delete a file. Returns { "success": true }.

FileObject:

Field	Required	Notes
`id`	No	File ID used in FileRef and collection operations.
`filename`	No	Original filename.
`hash`	No	Content hash.
`data.status`	No	Extraction status: "pending" until processed, then "completed".
`meta`	No	name, content_type, size.
`created_at / updated_at`	No	Unix timestamps.

4.3 Document collections

Method	Path	Purpose
POST	/document-collections	Create a collection (CollectionCreateRequest).
GET	/document-collections	List collections (CollectionList: items[], total).
GET	/document-collections/{id}	Get a collection including its files (CollectionWithFiles).
DELETE	/document-collections/{id}	Delete a collection.
POST	/document-collections/{id}/file/add	Attach a file (body: FileIdForm { file_id }).
POST	/document-collections/{id}/file/remove	Detach a file (body: FileIdForm { file_id }).

Create request and collection objects:

Field	Required	Notes
name	**Yes**	Collection name (CollectionCreateRequest).
description	No	Free-text description.
data	No	Free-form data object.
access_grants[]	No	Sharing rules. Each AccessGrant has principal_type, principal_id and permission.
meta	No	Returned on Collection; nullable.
files[]	No	Returned on CollectionWithFiles only.
write_access	No	Returned on list items and CollectionWithFiles: whether the caller can modify the collection.

5. Tools (MCP servers)

The Tools group is a managed MCP gateway. You can register your own MCP servers (scoped to your team), find servers and tools that are already available, and connect to them with any standard MCP client. Behind the scenes, each server is provisioned as a gateway and virtual server in IBM ContextForge and connected to OpenWebUI.

Listing shows all public servers plus your team's own servers. Servers that need credentials you have not connected are left out.

5.1 Endpoints

Method	Path	Purpose
GET	/tools	List MCP servers. Query: return_tools (bool, default false) to include each server's tools.
GET	/tools/search	Search servers and tools. Query: q (required, case-insensitive substring), page (default 0), size (default 20, max 100).
POST	/tools	Register a team-scoped MCP server. Returns 201.
PUT	/tools/{id}	Update display metadata or credentials (team-owned only).

Method	Path	Purpose
DELETE	/tools/{id}	Delete a team-owned server and tear down its gateway. Returns { "status": "deleted" }.
POST	/tools/{cfServerId}/mcp	MCP streamable-HTTP transport (JSON-RPC 2.0).
GET	/tools/{cfServerId}/mcp	MCP server-to-client SSE stream for an established session.

5.2 Connecting an MCP client

Each server in GET /tools includes a ready-to-use connection object. Paste it into a standard MCP client (MCP Inspector, Claude Code or Claude Desktop, opencode) and replace the placeholder with your developer API key:

```
"connection": {
  "type": "http",
  "url": "<connection.url from GET /tools>",
  "headers": { "Authorization": "Bearer <YOUR_ICA_API_KEY>" }
}
```

The /mcp endpoints are transparent: clients send JSON-RPC 2.0 calls (`initialize`, `tools/list`, `tools/call`) and get back either a single JSON body or an SSE stream. The `Mcp-Session-Id` header returned by initialize is sent back on later calls and on the GET stream. The documented example uses protocol version 2025-03-26. The cfServerId should be treated as an opaque value taken from connection.url.

5.3 Registering a server

CreateToolServerRequest (example: bearer-token auth):

```
POST /tools
{
  "name": "My Jira Server",
  "upstreamUrl": "https://mcp.example.com/jira/mcp",
  "description": "Jira issue tracker",
  "authType": "bearer",
  "authToken": "my-secret-token"
}
```

Field	Required	Notes
name	**Yes**	Display name.
upstreamUrl	**Yes**	URL of the upstream MCP server. Cannot be changed later.
description	No	Free-text description.
transport	No	Upstream transport type.
authType	No	e.g. "bearer"; OAuth is also supported.
authToken / authUsername / authPassword / authHeaders	No	Credentials for the chosen auth type.
oauthConfig	No	OAuth settings, when authType is OAuth.

The response (CreateToolServerResponse) returns id, name, status, upstreamUrl, authType, toolCount, url, authorizationUrl and tools[]. For non-OAuth servers the upstream is connected right away: tools are discovered and the status is `active`. For OAuth servers the status is `pending_authorization` and the user must open the returned `authorizationUrl` to finish the flow.

Updates via PUT are limited to display metadata (name, description, status) and credentials. Structural fields (upstreamUrl, source, ownerTeamId, ContextForge IDs) cannot be changed. To change them, delete the server and register it again.

6. Data model index

Area	Schemas
Chat	ChatCompletionRequest, ChatMessage, FileRef, ChatCompletionResponse, ChatCompletionChoice, Usage, Source
Model listing	Model, ModelsResponse
Files	FileObject, FileIdForm
Collections	CollectionCreateRequest, Collection, CollectionWithFiles, CollectionListItem, CollectionList, AccessGrant
Tools (MCP)	McpServer, McpTool, CreateToolServerRequest, CreateToolServerResponse, UpdateToolServerRequest
Common	Error